

UNIVERSIDADE ESTADUAL DE MARINGÁ
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA
CURSO DE ENGENHARIA DE SOFTWARE

INGRID LOUISE PEREIRA LOHMANN

COMPUTAÇÃO DE FUNÇÕES TRANSCENDENTAIS COM
APLICAÇÃO DE ROTAÇÃO DE COORDENADAS

Maringá
2026

INGRID LOUISE PEREIRA LOHMANN

COMPUTAÇÃO DE FUNÇÕES TRANSCENDENTAIS COM
APLICAÇÃO DE ROTAÇÃO DE COORDENADAS

Trabalho de Conclusão de Curso submetido ao curso de Engenharia de Software do Centro de Tecnologia da Universidade Estadual de Maringá como requisito parcial para a obtenção do título de Bacharel em Engenharia de Software.

Orientador(a): Prof. Airton Marco Polidório, Dr.

1 INTRODUÇÃO

A avaliação precisa e eficiente de funções matemáticas elementares, tais como seno, cosseno, exponencial, logaritmo e raiz quadrada, constitui uma operação fundamental em processamento digital de sinais, computação gráfica, simulações científicas e sistemas de controle em tempo real. Em processadores de uso geral contemporâneos, essas rotinas são suportadas por unidades de ponto flutuante (*Floating-Point Units*, FPUs) dotadas de circuitos aritméticos dedicados e conjuntos de instruções vetoriais, os quais executam operações de multiplicação e adição fundida (*Fused Multiply-Add*, FMA) em poucos ciclos de máquina.

Em contraste com esse cenário, nós de sensoramento de baixo consumo, dispositivos de Internet das Coisas (IoT), microcontroladores de arquitetura simplificada e circuitos integrados reconfiguráveis do tipo FPGA (*Field-Programmable Gate Array*) frequentemente operam sob restrições severas de área de silício e dissipação térmica. Nesses sistemas dedicados, a inclusão de circuitos multiplicadores combinacionais resulta em custos elevados. Consequentemente, métodos tradicionais de aproximação analítica, como expansões em séries de Taylor ou polinômios minimax de Chebyshev, tornam-se desvantajosos por demandarem múltiplos estágios encadeados de multiplicações de alta resolução.

Como alternativa algorítmica a essa limitação física, Jack E. Volder formulou em 1959 o algoritmo CORDIC (COordinate Rotation Digital Computer). O método viabiliza o cálculo de transformações de coordenadas e funções trigonométricas por meio de rotações vetoriais iterativas no plano cartesiano, utilizando unicamente operações de deslocamento binário (*shifts*), adições, subtrações e consultas a tabela pré-computadas de pequenas dimensões. Posteriormente, Walther (1971) unificou o algoritmo, expandindo sua formulação para abranger coordenadas circulares, hiperbólicas e lineares sob uma mesma base em equações de diferenças finitas.

Embora o CORDIC possua extensa documentação votada à síntese de circuitos digitais, sua implementação e caracterização em software sob formato de ponto flutuante de precisão dupla demandam considerações analíticas rigorosas. Entre os desafios técnicos pertinentes, destacam-se a garantia de conformidade com a norma IEEE 754-2019 para tratamento de valores excepcionais (NaN e infinitos), o desenvolvimento de rotinas de redução periódica de argumento para permitir a cobertura de todos os quadrantes trigonométricos, o tratamento numérico de zonas assintóticas e inserção correta de iterações repetidas no domínio hiperbólico para assegurar a convergência da sequência de aproximações.

O objetivo principal deste trabalho é projetar, implementar modularmente e caracterizar experimentalmente uma biblioteca unificada do algoritmo CORDIC em precisão dupla (64 bits), em conformidade com o padrão C99 e as diretrizes do padrão IEEE 754-2019. O desenvolvimento compreende:

- a) a implementação dos núcleos aritméticos nos modos de rotação e vetorização para coordenadas circulares e hiperbólicas;
- b) a síntese das rotinas transcendentais derivadas de exponencial, logaritmo natural, raiz quadrada e potenciação arbitrária;
- c) a inclusão de rotinas de redução periódica de domínio por quadrantes e o chaveamento por octantes via cotangente para a função tangente;
- d) a validação de limites de representação e conformidade com o padrão IEEE 754-2019;
- e) a caracterização experimental da taxa de convergência iterativa frente ao limitante analítico 2^{-N} ;
- f) o mapeamento da distribuição do erro absoluto no primeiro quadrante e a mensuração empírica da latência temporal frente às rotinas nativas da biblioteca `math.h`.
- g) a implementação dos núcleos aritméticos nos modelos de rotação e vetorização para as coordenadas circular e hiperbólica;
- h) a síntese de rotinas derivadas para cálculo de exponencial, logaritmo natural e raiz quadrada;
- i) a incorporação de mecanismos de redução analítica de argumento e propagação determinística de exceções numéricas;
- j) a validação sistemática de casos de borda e limites de representação;
- k) a caracterização da taxa de convergência do erro absoluto frente ao limitante analítico de contração 2^{-N} ;
- l) o mapeamento estatístico do erro ao longo do primeiro quadrante e quantificação comparativa da latência de execução frente às rotinas nativas da biblioteca `math.h`.

Com o propósito de dar subsídios ao leitor, este documento está estruturado em seis seções primárias. A Seção 2 apresenta a revisão bibliográfica e os fundamentos teóricos do CORDIC, das aproximações polinomiais e da representação numérica IEEE 754. A Seção 3 documenta o ambiente experimental e os procedimentos metodológicos adotados. A Seção 4 descreve a arquitetura modular da biblioteca desenvolvida e as rotinas internas. A Seção 5 expõe os resultados experimentais obtidos e suas respectivas discussões. A Seção 6 consolida as conclusões do estudo e aponta direcionamentos para trabalhos futuros. Por fim, os Apêndices apresentam a íntegra dos códigos-fonte desenvolvidos.

2 REVISÃO BIBLIOGRÁFICA

2.1 O ALGORITMO CORDIC CIRCULAR

O método CORDIC foi concebido originalmente por Volder (1959) para realizar a conversão de coordenadas e avaliação trigonométrica em computadores digitais embarcados de navegação aérea. O princípio analítico do algoritmo fundamenta-se na decomposição de uma rotação angular arbitrária em uma sequência de micropassos angulares fixos e pré-computados. No espaço euclidiano bidimensional, a rotação de um vetor inicial $V_0 = [x_0, y_0]^T$ por um ângulo θ produz o vetor rotacionado $V' = [x', y']^T$, conforme expresso matricialmente por:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} \quad (1)$$

Fatorando o termo $\cos \theta$, a relação (1) assume a seguinte forma equivalente:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \cos \theta \begin{bmatrix} 1 & -\tan \theta \\ \tan \theta & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} \quad (2)$$

Volder propôs decompor o ângulo total θ em uma soma ponderada de ângulos elementares α_i , tal que $\theta = \sum_{i=0}^{N-1} d_i \alpha_i$, onde $d_i \in \{-1, +1\}$ estabelece o sentido da rotação (horária ou anti-horária). Ao fixar a condição na qual a tangente de cada ângulo elementar seja dada por uma potência inteira negativa de dois:

$$\tan(\alpha_i) = 2^{-i} \implies \alpha_i = \arctan(2^{-i}) \quad (3)$$

a operação de multiplicação pela matriz de micropasso em cada iteração i é reduzida a operações de deslocamento aritmético à direita (*shift*) e adições. Substituindo a restrição (3) na expressão (2) e introduzindo uma variável de estado acumuladora de ângulo z , obtém-se o sistema de equações finitas:

$$x_{i+1} = x_i - d_i \cdot y_i \cdot 2^{-i} \quad (4)$$

$$y_{i+1} = y_i + d_i \cdot x_i \cdot 2^{-i} \quad (5)$$

$$z_{i+i} = z_i - d_i \cdot a_i \quad (6)$$

O fator residual $\cos(\alpha_i) = \frac{1}{\sqrt{1+2^{-2i}}}$ é simétrico em relação ao sinal d_i , visto que $[\cos(\alpha_i)]^2 = [\cos(-\alpha_i)]^2$. Dessa forma, para um total prefixado de N iterações, o ganho de escala acumulado K_N converge para uma constante independente dos sentidos de rotação adotados:

$$A_N = \prod_{i=0}^{N-1} \sqrt{1 + 2^{-2i}} \quad (7)$$

Quando $N \rightarrow \infty$, a magnitude do vetor sofre um ganho acumulado que converge para $A \approx 1,646760258121$. Para as componentes finais refletirem a rotação unitária sem necessidade de multiplicações de ajuste ao término do processo, pré-multiplica-se a coordenada inicial pelo inverso desse ganho, configurando $x_0 = K = 1/A_N \approx 0,607252935008881$ Parhami (2010).

O algoritmo admite dois modos de operações distintos:

- a) Modo de Rotação: o objetivo é conduzir a variável angular z a zero ($z_N \rightarrow 0$), adotando $d_i = \text{sgn}(z_i)$. Inicializando o sistema com $x_0 = K$, $y_0 = 0$ e $z_0 = \theta$, obtém-se $x_N = \cos \theta$ e $y_N = \sin \theta$.
- b) Modo de Vetorização: conduz a componente vertical y a zero ($y_N \rightarrow 0$), adotando $d_i = -\text{sgn}(y_i)$. Partindo de $[x_0, y_0]^T$ com $x_0 > 0$ e $z_0 = 0$, o registrador angular acumula $z_N = \arctan(y_0/x_0)$, enquanto a componente horizontal fornece a norma euclidiana amplificada pelo ganho da rotação, $x_N = A_N \sqrt{x_0^2 + y_0^2}$.

2.2 GENERALIZAÇÃO UNIFICADA DE WALTHER

Walther (1971) estendeu o formalismo de Volder introduzindo um parâmetro métrico de curvatura $m \in \{1, 0, -1\}$, formulando um conjunto unificado de equações aplicável a geometrias circulares, lineares e hiperbólicas:

$$x_{i+i} = x_i - m \cdot d_i \cdot y_i \cdot 2^{-S(m,i)} \quad (8)$$

$$y_{i+i} = y_i + d_i \cdot x_i \cdot 2^{-S(m,i)} \quad (9)$$

$$z_{i+i} = z_i - d_i \cdot \alpha_{(m,i)} \quad (10)$$

onde $S(m, i)$ representa a sequência de deslocamentos binários aplicados na iteração i , e $\alpha_{m,i}$ define o ângulo elementar associado ao espaço métrico selecionado. A Tabela 1 resume o espaço funcional unificado estabelecido por Walther.

Tabela 1: Espaço funcional unificado do algoritmo CORDIC segundo Walther

Parâmetro m	Sistema métrico	Ângulo elementar $a_{m,i}$	Funções	
			(Modo Ro- tação)	(Modo Veto- rização)
$m = 1$	Circular	$\arctan(2^{-i})$	$\sin \theta, \cos \theta$	$\arctan(y/x), \sqrt{x^2 + y^2}$
$m = 0$	Linear	2^{-i}	Multiplicação ($x \cdot z$)	Divisão (y/x)
$m = -1$	Hiperbólico	$\operatorname{arctanh}(2^{-i})$	$\sinh \theta, \cosh \theta, \exp \theta$	$\operatorname{arctanh}(y/x), \ln(u), \sqrt{x}$

Fonte: Adaptado de Walther (1971).

No domínio hiperbólico ($m = -1$), a condição de convergência do algoritmo não é satisfeita pela sequência monótona de índices inteiros $i = 1, 2, 3, \dots$ em decorrência da relação angular das funções hiperbólicas inversas. Para assegurar que o intervalo de convergência não apresente descontinuidades, Walther (1971) demonstrou analiticamente a necessidade de executar repetições em passos selecionados da sequência, aspecto também sistematizado por Muller (2016).

Na formulação original proposta por Walther, se uma iteração k foi repetida, a próxima repetição obrigatória ocorre no passo $3k + 1$, com a primeira ocorrência fixada em $k = 4$. De forma analítica equivalente, o j -ésimo índice de repetição i_j pode ser expresso em forma fechada por:

$$i_j = \frac{3^{j+1} - 1}{2}, \quad \text{para } j \geq 1 \quad (11)$$

A Equação (11) gera a sequência de passos $i \in \{4, 13, 40, 121, \dots\}$. Como a biblioteca desenvolvida neste estudo emprega um total de $N = 24$ iterações, apenas os índices correspondentes a $j = 1$ ($i_1 = 4$) e $j = 2$ ($i_2 = 13$) pertencem ao intervalo avaliado,

totalizando 26 passos efetivos de rotação e exigindo o ajuste do fator de escala acumulado inverso para $K_{\text{hip}} \approx 1,207497$.

Conforme destacado na revisão histórica conduzida por Meher *et al.* (2009), a modularidade estrutural do CORDIC assegurou sua longevidade em projetos de arquitetura de computadores. O algoritmo migrou das calculadoras de bolso para coprocessadores dedicados, processadores de sinais e núcleos de transformada rápida de Fourier. Andraka (1998) complementa que a ausência de multiplicadores físicos consolida o CORDIC como uma solução altamente adequada para circuitos lógicos programáveis, nos quais a densidade de portas lógicas constitui restrição primária do projeto.

2.3 MÉTODOS DE APROXIMAÇÃO POLINOMIAL E A REDUÇÃO POR OCTANTES

Diferentemente da abordagem vetorial iterativa do CORDIC, as bibliotecas matemáticas padrão em processadores de uso geral fundamentam-se prioritariamente em aproximações polinomiais e racionais otimizadas. Segundo Cody e Waite (1980), o cálculo eficiente de funções elementares envolve três etapas bem definidas: a redução de argumento de entrada para um domínio restrito, a aproximação em torno da origem e a reconstrução do valor sinal, considerando o sinal e o quadrante. Para as funções trigonométricas, Cody e Waite (1980) propõe-se particionar o domínio fundamental em octantes de largura $\pi/4$. No caso específico da tangente, quando o argumento θ ultrapassa $\pi/4$ e se aproxima de $\pi/2$, a função se aproxima de sua assíntota vertical, onde o cosseno tende a zero. Para evitar a instabilidade numérica de dividir um número por outro muito próximo de zero nessa região, os autores recomendam recorrer à identidade da cotangente sobre o argumento complementar $\alpha = \frac{\pi}{2} - \theta$:

$$\tan \theta = \cot\left(\frac{\pi}{2} - \theta\right) = \frac{\cos \alpha}{\sin \alpha} = \frac{1}{\tan \alpha} \quad (12)$$

Com essa transformação, o núcleo de cálculo opera sempre dentro do intervalo restrito $[0, \pi/4]$, mantendo as componentes intermediárias afastadas de valores nulos.

Essa estratégia foi adotada por Ng (1995) no desenvolvimento da biblioteca FDLIBM (*Freely Distributable Math Library*), cuja rotina `__kernel_tan` utiliza um argumento inteiro que indica o octante. Quando o ângulo está em um octante ímpar, a rotina calcula o valor no domínio reduzido e devolve o seu recíproco negativo, garantindo estabilidade numérica nos sistemas operacionais baseados em Unix e Linux.

2.4 ARITMÉTICA DE PONTO FLUTUANTE E O PADRÃO IEEE 754

O padrão IEEE 754 regulamenta a representação binária e as operações aritméticas em ponto flutuante ((Institute of Electrical and Electronics Engineers, 2019)). No formato de precisão dupla (64 bits), qualquer número real normalizado x é expresso por:

$$x = (-1)^s \times 2^{e-1023} \times (1.f_1f_2\dots f_{52}) \quad (13)$$

onde s é o bit de sinal, e representa o expoente polarizado de 11 bits e os 52 bits restantes formam a mantissa fracionária, fornecendo uma precisão de 53 bits (aproximadamente 16 dígitos decimais significativos).

Conforme discutido por Goldberg (1991), o cálculo em aritmética finita introduz erros inevitáveis de arredondamento por quantização da mantissa, além de erros por cancelamento catastrófico na subtração de valores de magnitudes semelhantes.

Kahan (1987) analisou o tratamento de singularidades e cortes de ramificação em ponto flutuante, mostrando que imprecisões no argumento propagam desvios severos em regiões de assíntota vertical. Ao avaliar a tangente diretamente pela razão $\tan \theta = \frac{\sin \theta}{\cos \theta}$, a sensibilidade em relação a um resíduo numérico Δc presente no cosseno pode ser aproximada pela derivada:

$$|\Delta(\tan \theta)| \approx \left| \frac{\partial}{\partial \cos \theta} \left(\frac{\sin \theta}{\cos \theta} \right) \right| \Delta c = \frac{|\sin \theta|}{\cos^2 \theta} \Delta c \quad (14)$$

À medida que $\theta \rightarrow \frac{\pi}{2}$, o denominador tende a zero ($\cos \theta \rightarrow 0$) e o fator de sensibilidade $\frac{1}{\cos^2 \theta}$ cresce de forma acentuada. Muller (2016) ressalta que qualquer resíduo finito proveniente de truncamento anterior é amplificado em várias ordens de grandeza pela divisão. Por esse motivo, Kahan (1987) e Muller (2016) indicam que se deve evitar a divisão direta $\sin / \cos$ nas proximidades de singularidades, adotando a relação complementar da cotangente.

3 METODOLOGIA E AMBIENTE EXPERIMENTAL

3.1 PLATAFORMA DE EXECUÇÃO E COMPILAÇÃO

Os ensaios experimentais foram realizados em contêiner Linux operando sobre um processador AMD EPYC 9v74. As opções de compilação foram mantidas constantes em todos os testes. A Tabela 2 apresenta as especificações de hardware e software utilizadas

Tabela 2: Especificações da plataforma experimental de hardware e software

Componente	Especificação Técnica
Processador	AMD EPYC 9V74, arquitetura x86_64, 80 núcleos
Frequência nominal da CPU	2,87 GHz (2871,294 MHz)
Recursos computacionais alocados	2 núcleos lógicos (vCPUs) e 1024 KB de memória cache L3
Memória RAM	7,8 GiB
Conjunto de instruções	AVX, AVX2, FMA e SSE2
Sistema operacional	Alpine Linux v3.23, kernel Linux 6.8.0-1064-azure
Biblioteca padrão C	musl libc versão 1.2.5
Compilador C	GCC versão 15.2.0
Opções de compilação	<code>-Wall -Wextra -Werror -std=c11 -Iinclude -lm</code>

Fonte: elaborada pela autora.

Todas as rotinas foram compiladas no padrão C11 com checagem estrita ativa pelas opções `-Wall -Wextra -Werror`. A biblioteca matemática padrão `math.h` da musl libc serviu como referência comparativa para a validação dos resultados e as medidas de tempo.

3.2 PROTOCOLO EXPERIMENTAL E PROCEDIMENTOS DE TESTE

Com o objetivo de avaliar a exatidão numérica, os limites de convergência e o desempenho temporal da biblioteca implementada, foram estabelecidas quatro baterias de ensaios experimentais. Cada procedimento foi estruturado para isolar aspectos específicos do algoritmo e verificar a sua conformidade perante os requisitos aritméticos e arquiteturais estabelecidos neste trabalho.

3.2.1 Testes de Conformidade e Casos de Borda IEEE 754

O algoritmo CORDIC fundamenta-se em iterações de rotação contínuas, porém uma biblioteca de software destinada a sistemas embarcados requer tratamento rigoroso de exceções aritméticas. Por este motivo, foi elaborado um conjunto de testes voltado a verificar se as funções implementadas interceptam corretamente entradas não padroniza-

das antes do início dos laços iterativos.

Nesta bateria, foram avaliadas as seguintes condições de contorno:

- Identidades trigonométricas e hiperbólicas em zero, como $\sin(0)$, $\cos(0)$, $\tan(0)$, $\sinh(0)$ e $\cosh(0)$;
- Valores fundamentais em funções transcendentais, tais como $\exp(0)$, $\ln(1)$ e $\sqrt{0}$;
- Entradas fora do domínio analítico das funções, incluindo argumentos negativos em $\sqrt{x}$ e argumentos nulos ou negativos em $\ln(x)$;
- Propagação estrita de valores especiais previstos pelo padrão IEEE 754, como NaN e infinitos positivos e negativos ($\pm\infty$).

O critério de conformidade adota como referência os retornos estabelecidos pela biblioteca padrão `math.h`, verificando se a implementação garante estabilidade numérica sem gerar interrupções não tratadas durante a execução.

3.2.2 Ensaio de Convergência Iterativa

O objetivo deste ensaio é acompanhar experimentalmente a taxa de decaimento do erro absoluto das funções à medida que o número de iterações N é incrementado de 1 até 24. De acordo com a formulação teórica de Walther (1971), o método CORDIC exibe convergência linear, em que a aproximação angular ganha cerca de 1 bit de precisão a cada passo, o que resulta em um limite assintótico proporcional a 2^{-N} .

Para a realização deste ensaio, foram selecionados pontos fixos e representativos de cada modo de operação:

- Modo circular: cálculo de $\sin(\theta)$ e $\cos(\theta)$ no ponto $\theta = \pi/4$;
- Modo hiperbólico: cálculo de $\sinh(u)$ e $\cosh(u)$ no ponto $u = 0,5$;
- Funções transcendentais: cálculo de $\exp(x)$ em $x = 1$, $\ln(w)$ em $w = 2$ e $\sqrt{y}$ em $y = 2$.

Os pontos foram escolhidos dentro da faixa primária de convergência de cada modo, permitindo observar se a taxa de convergência acompanha o limite teórico 2^{-N} e identificar a iteração em que ocorre a saturação da precisão devido aos erros de truncamento em dupla precisão.

3.2.3 Teste de Estresse no Primeiro Quadrante

Para avaliar a estabilidade contínua das funções circulares em um domínio denso, foi conduzido um teste de estresse ao longo do intervalo $[0, \pi/2]$, utilizando uma amostragem com passo regular de 0,005 rad, resultando em 315 pontos de medição, incluindo o ponto inicial 0 e considerando os pontos até 1,57 rad

A escolha deste intervalo e desta densidade amostral apoia-se em três justificativas metodológicas:

- **Suficiência analítica por simetria:** no algoritmo CORDIC, ângulos situados fora do primeiro quadrante são submetidos a etapas prévias de redução de argumentos fundamentadas em inversões de sinal ou translações modulares. Dessa forma, as rotações do algoritmo e o acúmulo de erros de arredondamento concentram-se estritamente no intervalo $[0, \pi/2]$, tornando desnecessária a extensão da varredura densa para quadrantes complementares;
- **Verificação da transição em $\pi/4$:** O ponto central $\theta = \pi/4$ define a fronteira na qual o algoritmo chaveia entre a avaliação direta ($\sin \theta / \cos \theta$) e o emprego da identidade da cotangente ($\cos \alpha / \sin \alpha$). Uma amostragem contínua permite verificar se há continuidade na curva de erro ou se ocorrem descontinuidades numéricas nesta fronteira;
- **Mapeamento da assíntota vertical:** a proximidade de θ em relação a $\pi/2$ provoca o decaimento de $\cos \theta$ para valores próximos a zero, elevando o número de condição relativo da função tangente, dado analiticamente por $\frac{1}{\cos^2 \theta}$. O passo fino de 0,005 rad possibilita registrar a perda gradual de algarismos significativos decorrente do mau condicionamento da razão à medida que o limite analítico é aproximado.

3.2.4 Benchmark de Latência e Desempenho Temporal

O ensaio de latência visa comparar o tempo de execução e a quantidade de ciclos de clock exigidos pelas operações CORDIC em relação às funções da biblioteca `math.h`, compiladas com aceleração por instruções de hardware da FPU. O tempo decorrido foi obtido por meio da rotina `clock_gettime(CLOCK_MONOTONIC)`, enquanto a contagem de ciclos de clock foi registrada diretamente em nível de hardware por meio da instrução intrínseca `__rdtsc()` para a arquitetura x86_64.

Para contornar as limitações inerentes à medição de intervalos temporais em sistemas operacionais multitarefa, adotou-se um volume amostral de 1×10^6 repetições consecutivas para cada operação avaliada. Essa amostragem fundamenta-se nas seguintes características:

- **Resolução do temporizador do sistema:** as instruções nativas da FPU executam em ordens de grandeza de dezenas de nanossegundos. Medições isoladas sofriam influência do ruído e do overhead de chamada do temporizador `clock_gettime`. Ao acumular 1×10^6 iterações, o tempo total medido é deslocado para a escala de milissegundos e segundos, tornando a incerteza do temporizador desprezível;
- **Amortização de interrupções e preempção:** em ambientes multitarefa preemptivos, trocas de contexto e interrupções de hardware inserem ruídos estocásticos na

medição. A média obtida a partir de uma amostra ampla de execuções dilui essas variações, convergindo para o tempo característico do algoritmo;

- **Estabilização do estado de hardware:** A repetição contínua assegura que as instruções e tabelas de constantes permaneçam residentes na memória cache e que as previsões de desvio estabilizem, permitindo medir o desempenho do algoritmo em regime estacionário.
- **Prevenção de eliminação de código morto:** as saídas das operações executadas nos laços de repetição foram acumuladas em uma variável dotada do qualificador `volatile`, garantindo que o compilador não descarte as computações por otimização de fluxo de dados durante a geração do código binário.

4 PROPOSTA E IMPLEMENTAÇÃO DA BIBLIOTECA CORDIC

4.1 ARQUITETURA DE SOFTWARE E ESTRUTURA DE MÓDULOS

A biblioteca foi desenvolvida no padrão C11, adotando uma organização modular que segrega os cabeçalhos de interface, os núcleos iterativos de cálculo, os módulos transcendentais derivados e os utilitários de manipulação de ponto flutuante. As constantes analíticas fundamentais e os fatores de escala foram gerados previamente por meio de rotinas auxiliares dedicadas, evitando a inclusão de chamadas dinâmicas a funções trigonométricas no fluxo principal de execução.

A Tabela 3 sintetiza a organização dos diretórios e detalha a responsabilidade técnica atribuída a cada arquivo do projeto.

Tabela 3: Estrutura de diretórios e arquivos da biblioteca

Diretório	Arquivo	Responsabilidade técnica
tools/	build_cordic_table.c	Geração e exportação estática das tabelas de busca e dos fatores de escala
include/	cordic_types.h	Definição da estrutura de dados <code>CordicVector</code>
include/	cordic_lut.h	Tabelas de constantes de micropasso e constantes de escala pré-calculadas

Continua na próxima página

Tabela 3 – continuação

Diretório	Arquivo	Responsabilidade técnica
include/	cordic_circular.h	Declaração das rotinas de rotação, vetorização e funções circulares
include/	cordic_hyperbolic.h	Declaração dos núcleos hiperbólicos e controle de iterações repetidas
include/	cordic_transcendental.h	Declaração das funções derivadas: exponencial, logaritmo, raiz quadrada e potência
include/	ieee_utils.h	Definições de constantes numéricas, tipos em união IEEE 754 e macro de deslocamento
src/	cordic_circular.c	Implementação do núcleo circular, redução periódica e partição por octantes
src/	cordic_hyperbolic.c	Implementação dos núcleos hiperbólicos de rotação e vetorização
src/	cordic_transcendental.c	Implementação das funções transcendentais compostas e condicionamento numérico
src/	ieee_utils.c	Rotinas auxiliares para análise de campos binários em ponto flutuante

Fonte: elaborada pela autora.

A manipulação direta dos dados no nível de bits exigiu uma estrutura de representação compatível com o formato de dupla precisão da norma IEEE 754. No arquivo `include/ieee_utils.h`, estabeleceu-se o tipo `DoubleIEEE`, estruturado como uma união entre um valor em ponto flutuante convencional (`double`) e um mapeamento posicional de campos de bits (*bitfields*):

```

1 typedef union {
2     double x;
3     struct {

```

```

4         unsigned long long f : 52;
5         unsigned long long E : 11;
6         unsigned long long s : 1;
7     } bits;
8 } DoubleIEEE;

```

Com essa união, o vetor tridimensional característico das formulações CORDIC foi definido em `include/cordic_types.h` sob a estrutura `CordicVector`:

```

1 typedef struct {
2     DoubleIEEE x;
3     DoubleIEEE y;
4     DoubleIEEE z;
5 } CordicVector;

```

Os membros `x` e `y` armazenam as coordenadas da projeção vetorial no plano, enquanto `z` atua como o acumulador angular no modo circular ou residual no modo hiperbólico.

A operação elementar de divisão por potências inteiras de dois (2^{-i}), obrigatória a cada passo i , foi implementada por manipulação do expoente binário polarizado por meio da macro `CORDIC_SHIFT_DOUBLE(val, i)`. Essa operação subtrai o índice i do campo de 11 bits do expoente E :

```

1 #define CORDIC_SHIFT_DOUBLE(val, i) do { \
2     if ((val).bits.E > (unsigned int)(i)) { \
3         (val).bits.E -= (i); \
4     } else { \
5         (val).bits.E = 0; \
6         (val).bits.f = 0; \
7     } \
8 } while(0)

```

Como o campo E opera com inteiros sem sinal, subtrações diretas em magnitudes pequenas poderiam provocar estouro negativo (*underflow*), fazendo com que o valor de E assumisse o limite máximo (2047) e fosse interpretado incorretamente pelo hardware como infinito. A macro introduz uma verificação condicional: caso o expoente armazenado seja menor ou igual ao deslocamento pretendido ($E \leq i$), a mantissa e o expoente são anulados de imediato, garantindo truncamento seguro para zero sem desestabilizar os registradores aritméticos.

As tabelas de consulta e as constantes analíticas foram consolidadas no arquivo `include/cordic_lut.h` pelo programa `tools/build_cordic_table.c`. Para $N = 24$

iterações, a constante de escala circular inversa $K_{\text{circ}} = \prod_{i=0}^{23} (1 + 2^{-2i})^{-1/2}$ resultou em:

$$K_{\text{circ}} \approx 0,607252935008883 \quad (15)$$

No domínio hiperbólico, considerando o início em $i = 1$ e as repetições mandatórias nos índices 4 e 13, o fator de compensação acumulado $K_{\text{hip}} = \prod_i (1 - 2^{-2i})^{-1/2}$ resultou no valor:

$$K_{\text{hip}} \approx 1,207497067763071 \quad (16)$$

Os vetores constantes `CORDIC_CIRCULAR_LUT[24]` e `CORDIC_HYPERBOLIC_LUT[25]` armazenam os ângulos de rotação elementar $\arctan(2^{-i})$ e $\operatorname{arctanh}(2^{-i})$, calculados em precisão estendida de 15 casas decimais.

4.2 MÓDULO CIRCULAR E CONDICIONAMENTO DE DOMÍNIO

O núcleo circular de rotação foi implementado na função `cordic_circular_rotate` contida em `src/cordic_circular.c`. A cada iteração, as componentes do vetor sofrem deslocamentos parametrizados via `CORDIC_SHIFT_DOUBLE` e são atualizadas de acordo com o sinal do registrador angular z :

```

1 CordicVector cordic_circular_rotate(CordicVector v, int iterations) {
2     for (int i = 0; i < iterations; i++) {
3         DoubleIEEE x_shift = v.x;
4         DoubleIEEE y_shift = v.y;
5
6         CORDIC_SHIFT_DOUBLE(x_shift, i);
7         CORDIC_SHIFT_DOUBLE(y_shift, i);
8
9         if (v.z.x >= 0.0) {
10             v.x.x = v.x.x - y_shift.x;
11             v.y.x = v.y.x + x_shift.x;
12             v.z.x = v.z.x - CORDIC_CIRCULAR_LUT[i];
13         } else {
14             v.x.x = v.x.x + y_shift.x;
15             v.y.x = v.y.x - x_shift.x;
16             v.z.x = v.z.x + CORDIC_CIRCULAR_LUT[i];
17         }
18     }
19     return v;
20 }
```

No modo de vetorização (`cordic_circular_vector`), as operações direcionam-se a anular a componente y ($y \rightarrow 0$), empregando a condição $v.y.x < 0.0$ para a escolha do

sentido do micropasso. Esse procedimento permite avaliar o arco-tangente ao inicializar o vetor com $x_0 = 1$, $y_0 = \text{valor}$ e $z_0 = 0$, resultando no ângulo acumulado em z ao término de 24 iterações.

A faixa de convergência angular das 24 iterações circulares é restrita a $\theta_{\max} = \sum_{i=0}^{23} \arctan(2^{-i}) \approx 1,743287 \text{ rad } (\approx 99,88^\circ)$. Para permitir a aplicação de ângulos arbitrários, implementou-se em `cordic_circular_sin_cos` um esquema de redução periódica e mapeamento de quadrantes:

1. Entradas classificadas como não numéricas (NaN) ou infinitas geram retorno imediato de NaN, atendendo aos requisitos da norma IEEE 754-2019;
2. O argumento θ é reduzido ao período fundamental $[-\pi, \pi]$ mediante a aplicação da função `fmod(angle_rad, TWO_PI)`;
3. Quando o ângulo resultante situa-se no segundo ou terceiro quadrantes ($|\theta| > \pi/2$), aplica-se a simetria de meia-onda:

$$\theta_{\text{red}} = \theta \mp \pi, \quad \text{sinal} = -1,0 \quad (17)$$

conduzindo o argumento para o intervalo $[-\pi/2, \pi/2]$. O vetor inicial é então carregado com $x_0 = K_{\text{circ}}$, $y_0 = 0$ e $z_0 = \theta_{\text{red}}$, e os valores finais de cosseno e seno recebem o sinal correspondente ao quadrante original.

Na função tangente (`cordic_circular_tan`), a divisão direta $\sin \theta / \cos \theta$ perde estabilidade nas vizinhanças de $\pi/2$ em razão do crescimento de $1/\cos^2 \theta$, o que amplifica os erros residuais de arredondamento. Para assegurar precisão em todo o primeiro quadrante, a rotina adota o chaveamento por octantes fundamentado nas identidades descritas por Cody e Waite (1980):

1. Explora-se a paridade ímpar $\tan(-\theta) = -\tan \theta$, tratando apenas ângulos positivos com controle de sinal;
2. O argumento é reduzido modularmente ao intervalo $[0, \pi)$ e, caso recaia no segundo quadrante ($\theta > \pi/2$), projeta-se sobre o primeiro quadrante via $\pi - \theta$, invertendo o sinal;
3. A proximidade imediata da assíntota é verificada por $|\theta - \pi/2| < 10^{-15}$, retornando infinito com o respectivo sinal analítico;
4. Se $\theta \leq \pi/4$, a avaliação ocorre de forma direta mediante a razão $\sin \theta / \cos \theta$;
5. Se $\theta > \pi/4$, calcula-se o ângulo complementar $\alpha = \pi/2 - \theta$, o qual recai necessariamente em $[0, \pi/4]$, e obtém-se a tangente pela identidade da cotangente:

$$\tan \theta = \cot(\alpha) = \frac{\cos \alpha}{\sin \alpha} \quad (18)$$

Esse desvio impede que o denominador assuma valores excessivamente pequenos dentro do núcleo iterativo, mantendo as grandezas intermediárias equilibradas.

4.3 MÓDULO HIPERBÓLICO E FUNÇÕES DERIVADAS

O cálculo no domínio hiperbólico foi implementado no arquivo `src/cordic_hyperbolic.c` sob o parâmetro métrico $m = -1$. Os núcleos de rotação e vetorização foram particionados em passos elementares (`cordic_hyperbolic_rotate_step` e `cordic_hyperbolic_vector_step`). Para satisfazer os critérios de convergência analítica da base binária, as iterações nos índices $i = 4$ e $i = 13$ são obrigatoriamente duplicadas:

```

1 CordicVector cordic_hyperbolic_rotate(CordicVector v, int iterations) {
2     for (int i = 1; i <= iterations; i++) {
3         v = cordic_hyperbolic_rotate_step(v, i);
4         if (i == 4 || i == 13) {
5             v = cordic_hyperbolic_rotate_step(v, i);
6         }
7     }
8     return v;
9 }

```

A rotina `cordic_hyperbolic_sinh_cosh` inicializa o vetor com $x_0 = K_{\text{hip}}$, $y_0 = 0$ e $z_0 = \theta$, convergindo simultaneamente para $\cosh \theta$ em x e $\sinh \theta$ em y . A tangente hiperbólica é avaliada pela razão entre as duas saídas, enquanto a função inversa $\operatorname{arctanh}(u)$ opera por vetorização com entradas restritas ao intervalo aberto $(-1, 1)$, retornando `NAN` para argumentos nas fronteiras ou fora do domínio.

Com base no motor hiperbólico, o arquivo `src/cordic_transcendental.c` implementa as quatro rotinas transcendentais compostas:

a) **Função Exponencial** (`cordic_exp`): para evitar a perda de convergência em magnitudes elevadas de x , o argumento é reduzido em múltiplos de $\ln 2$:

$$x = k \cdot \ln 2 + r, \quad k = \operatorname{round}\left(\frac{x}{\ln 2}\right) \quad (19)$$

A constante $\ln 2$ é determinada internamente pelo próprio CORDIC através da identidade $\ln 2 = 2 \cdot \operatorname{arctanh}(1/3)$. O termo residual r situa-se na faixa primária de convergência, permitindo obter $e^r = \cosh r + \sinh r$. O valor final é recomposto diretamente pelo ajuste do expoente em ponto flutuante por meio da função nativa `ldexp(exp_r, k)`.

b) **Logaritmo Natural** (`cordic_ln`): entradas nulas retornam $-\infty$, enquanto argumentos negativos geram `NAN`. Para valores positivos, a entrada é decomposta pela rotina `frexp(x, &exp_val)`, isolando a mantissa normalizada $m \in [0.5, 1.0)$ e o expoente inteiro associado. Aplica-se a transformação analítica:

$$u = \frac{m - 1}{m + 1} \quad (20)$$

Avaliando-se $\operatorname{arctanh}(u)$ por vetorização hiperbólica, deduz-se $\ln m = 2 \cdot \operatorname{arctanh}(u)$.

A reconstrução do resultado é dada pela combinação linear:

$$\ln x = \ln m + \exp_val \cdot \ln 2 \quad (21)$$

c) **Raiz Quadrada** (`cordic_sqrt`): entradas negativas geram NAN, e a raiz de zero retorna zero. Para entradas positivas, extrai-se a mantissa m via `frexp`. Caso o expoente seja ímpar, ajusta-se $m \leftarrow 2m$ para manter a potência de dois como um múltiplo par de quatro (4^k), garantindo que a raiz do termo exponencial seja uma potência exata de dois (2^k). Carrega-se o vetor CORDIC com:

$$x_0 = m + 0,25, \quad y_0 = m - 0,25, \quad z_0 = 0 \quad (22)$$

A invariância hiperbólica assegura que $x_0^2 - y_0^2 = m$. Ao término da vetorização, a componente x fornece $\sqrt{m}/K_{\text{hip}}$. Desfaz-se o ganho multiplicando por K_{hip} , e o resultado final é obtido por `ldexp(sqrt_m, k)`.

d) **Potenciação Arbitrária** (`cordic_pow`): avalia a base b e o expoente y mediante a identidade $b^y = \exp(y \ln b)$, integrando as rotinas `cordic_ln` e `cordic_exp`. A função trata diretamente as exceções analíticas para bases nulas ou negativas segundo o padrão IEEE 754.

5 RESULTADOS E DISCUSSÕES

Nesta seção são apresentados e discutidos os resultados experimentais obtidos a partir dos ensaios de validação de casos de borda, convergência iterativa do erro, teste de estresse no primeiro quadrante e medições de latência temporal frente à biblioteca padrão `math.h`.

REFERÊNCIAS

ANDRAKA, Ray. A Survey of CORDIC Algorithms for FPGA Based Computers. *In: PROCEEDINGS of the ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. Monterey: ACM, 1998. p. 191–200. DOI: 10.1145/275107.275139.

CODY, William J.; WAITE, William. SIN/COS and TAN/COT. *In: SOFTWARE Manual for the Elementary Functions*. Englewood Cliffs: Prentice-Hall, 1980. cap. 8–9, p. 125–173.

GOLDBERG, David. What Every Computer Scientist Should Know About Floating-Point Arithmetic. **ACM Computing Surveys**, v. 23, n. 1, p. 5–48, mar. 1991. DOI: 10.1145/103162.103163.

INSTITUTE OF ELECTRICAL AND ELECTRONICS ENGINEERS. **IEEE Standard for Floating-Point Arithmetic**: IEEE Std 754-2019 (Revision of IEEE 754-2008). Piscataway: [s. n.], 2019. DOI: 10.1109/IEEESTD.2019.8766229.

KAHAN, William. Branch Cuts for Complex Elementary Functions, or, Much Ado About Nothing’s Sign Bit. *In: ISERLES, Arie; POWELL, Michael (ed.). The State of the Art in Numerical Analysis*. Oxford: Clarendon Press, 1987. p. 165–211.

MEHER, Pramod Kumar *et al.* 50 Years of CORDIC: Algorithms, Architectures, and Applications. **IEEE Transactions on Circuits and Systems I: Regular Papers**, v. 56, n. 9, p. 1893–1907, set. 2009. DOI: 10.1109/TCSI.2009.2025803.

MULLER, Jean-Michel. The CORDIC Algorithm. *In: ELEMENTARY Functions: Algorithms and Implementation*. 3. ed. Boston: Birkhäuser, 2016. cap. 7, p. 181–216. Capítulo 7: The CORDIC Algorithm. DOI: 10.1007/978-1-4899-7983-4.

NG, Kwok Choi. **FDLIBM: Freely Distributable Math Library**. Versão 5.3. Sun Microsystems. 1995. Disponível em: <https://www.netlib.org/fdlibm/>. Acesso em: 25 jul. 2026.

PARHAMI, Behrooz. The CORDIC Algorithms. *In: COMPUTER Arithmetic: Algorithms and Hardware Designs*. 2. ed. New York: Oxford University Press, 2010. cap. 22, p. 441–458. Chapter 22: Coordinate Rotation Digital Computing (CORDIC).

VOLDER, Jack E. The CORDIC Trigonometric Computing Technique. **IRE Transactions on Electronic Computers**, EC-8, n. 3, p. 330–334, set. 1959. DOI: 10.1109/TEC.1959.5222693.

WALTHER, J. S. A Unified Algorithm for Elementary Functions. *In: PROCEEDINGS of the Spring Joint Computer Conference*. Atlantic City: AFIPS Press, 1971. p. 379–385. DOI: 10.1145/1478786.1478840.

APÊNDICE A – CÓDIGO DO BENCHMARK DE LATÊNCIA

```
1 #include <stdio.h>
2 #include <time.h>
3
4 int main(void) {
5     clock_t inicio = clock();
6
7     // Código do benchmark
8
9     clock_t fim = clock();
10
11     double tempo = (double)(fim - inicio) / CLOCKS_PER_SEC;
12
13     printf("Tempo: %f segundos\n", tempo);
14
15     return 0;
16 }
```